

# 40 Buoni Consigli da Rispettare

Tali suggerimenti, in linea di principio, non dovrebbero essere forniti, poiché è attraverso l'esperienza diretta, anche tramite l'errore e la difficoltà, che ciascuno costruisce progressivamente il proprio percorso di apprendimento. Tuttavia, in questa occasione, intendo offrire alcuni consigli immediati che possano fornire indicazioni utili per affrontare con successo l'esame.

Il corso in questione rappresenta uno degli insegnamenti che, una volta superati, lasciano un effettivo patrimonio di conoscenze: qualora l'esame sia stato affrontato con serietà e adeguato impegno, lo studente maturerà una solida preparazione di base.

Iniziamo.

---

1. È opportuno individuare un/una compagno/a di progetto disponibile a sostenere **carichi di lavoro intensivi**, inclusi periodi di lavoro prolungato nelle ore notturne (*se necessario*).
2. La relazione di progetto è consigliato che venga redatta in **lingua inglese**, poiché tale lingua si presta in modo particolarmente efficace alla descrizione e alla formalizzazione dei processi di progettazione software.
3. Nei requisiti funzionali **non deve mai essere utilizzata** l'espressione "**shall allow**".
4. È fortemente sconsigliato iniziare subito la fase di programmazione senza una chiara comprensione preliminare del sistema e procedere solo successivamente alla produzione dei diagrammi. **La stesura dei diagrammi deve precedere l'implementazione**: si tratta di un corso di progettazione, non di programmazione. Operare in senso inverso dimostra una carente capacità progettuale.
5. Negli activity diagram è opportuno/consigliabile utilizzare il timer all'interno delle strutture **fork-join**, rappresentando l'azione come un comportamento eseguito dal sistema.
6. L'uso del timer non deve essere casuale: esso va inserito esclusivamente quando risulta semanticamente corretto e giustificato dal modello. Mica va buttato così a caso.
7. Nella redazione delle User Stories è necessario prestare attenzione alla **punteggiatura**, in particolare all'uso corretto delle **virgole**.

8. Nei Sequence Diagram è **fondamentale** distinguere correttamente tra chiamate **sincrone** e **asincrone**, utilizzando le frecce appropriate. Errori di questo tipo comportano **significative penalizzazioni** in sede di valutazione (-10).

9. Le decisioni progettuali assunte nelle fasi iniziali del corso sono destinate a evolversi nel tempo: quanto stabilirete all'inizio **raramente** andrà a coincidere con la versione finale del progetto.

10. **Una conoscenza adeguata dei design pattern è imprescindibile.**

11. Non è consigliabile limitarsi all'uso di un numero minimo di design pattern: sebbene la documentazione fornita dai docenti indichi un requisito minimo di 2 (uno x ogni membro), un progetto correttamente progettato ne richiederà **naturalmente un numero maggiore**.

12. Negli activity diagram, una volta definita un'azione, l'azione successiva può essere formulata come "validate ..."; successivamente, è opportuno utilizzare il **nodo decisionale** (rombo) per **distinguere** i diversi casi possibili.

13. Il progetto deve presentare un **elemento di innovazione**: non è sufficiente limitarsi alla realizzazione di un semplice sistema gestionale. **È richiesta almeno una componente innovativa, in assenza della quale sono previste penalizzazioni in sede di valutazione.**

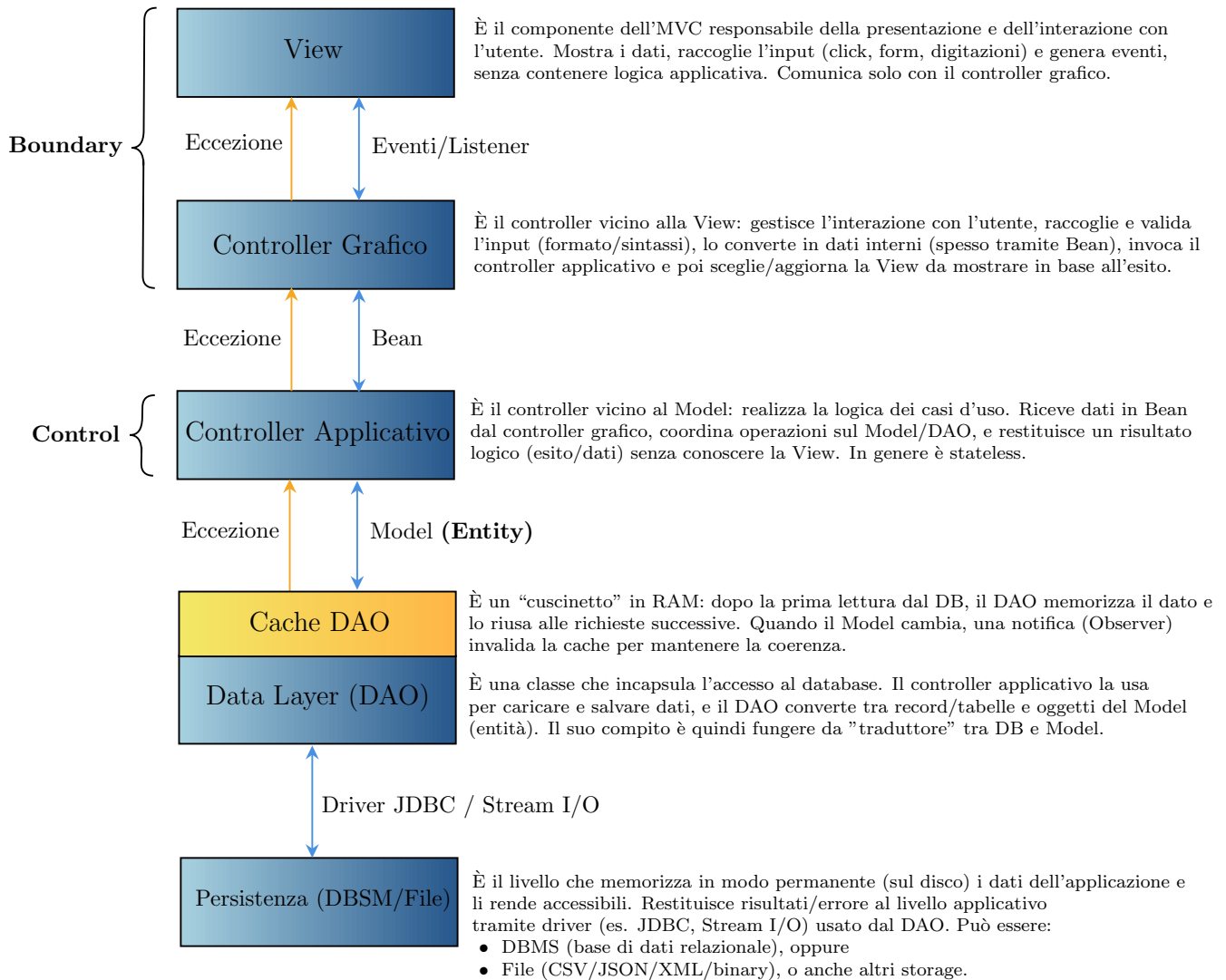
14. **Ogni contenuto inserito nella relazione contribuisce alla valutazione complessiva:** particolare attenzione deve essere riservata alla sintassi, alla punteggiatura e, più in generale, alla qualità della scrittura.

15. La **capacità di progettare in modo semplice un sistema complesso** è un elemento valutato positivamente. La complessità non deve tradursi in diagrammi troppo grossi o poco leggibili: l'obiettivo della progettazione è **chiarire e rendere comprensibile un sistema complesso**. Se pensate di aver fatto bene la progettazione perchè avete l'activity diagram gigantesco, state sbagliando.

16. Ai docenti non interessa il processo di creazione del database né il dettaglio della sua struttura: non è richiesto mostrare lo schema delle tabelle, le query utilizzate, né la definizione di chiavi primarie o esterne. **Non è un corso di Basi di Dati**. L'analisi si concentra sui controller grafici, sui controller applicativi, sulle loro interazioni con i model e sul livello di persistenza rappresentato dalle DAO; oltre questo livello, la valutazione non si spinge ulteriormente.

17. La progettazione richiede l'utilizzo dell'approccio MVC/MVP.  
Di seguito elencato:

## SCHEMA MVC/MVP - M. Marulli, S. Remoli



### Da sapere:

1. Le eccezioni possono nascere in basso (DAO/Model/Controller applicativo) e propagarsi "verso l'alto", vengono intercettate dal controller grafico e tradotte in un esito/messaggio che la View può mostrare all'utente.
2. Le Bean sono oggetti semplici di scambio dati tra controller grafico e controller applicativo; non sono Model. Controllano solo aspetti sintattici/formali dell'input/output (formato, campi obbligatori, parsabilità), non le regole di business. Non hanno costruttore (vuoto).
3. Con il disaccoppiamento tra Boundary (View + controller grafico) e Control (controller applicativo), se cambia l'interfaccia anche radicalmente (es. GUI → riga di comando), la logica applicativa resta invariata: si modifica solo la Boundary che raccoglie/mostra i dati e invoca gli stessi casi d'uso.
4. Il controller applicativo realizza e coordina gli internal steps del caso d'uso: orchestra le chiamate a Model e DAO nell'ordine corretto e produce l'esito logico, mentre la Boundary si limita a input e output.
5. È buona pratica non delegare tutta la logica al DB: la persistenza dovrebbe fare accessi "puliti" (letture/scritture semplici), mentre la logica sta soprattutto nel Model.
6. Lo schema grafico è coerente sia con MVC che con MVP, tuttavia:
  - MVC: serve un meccanismo tipo Observer per gestire la sincronizzazione Model ↔ View nella quale il Model può essere in *push* o in *pull*.
  - MVP: l'Observer non è necessario tra Model e View, perché la View non parla col Model: c'è il Presenter (che è di fatto "il controller") che riceve le azioni della View, aggiorna il Model e poi aggiorna direttamente la View.

18. È sconsigliato affidarsi a strumenti come **Gemini** o **ChatGPT** per la generazione di query complesse: in tal caso, infatti, si rischia di spostare la “logica” fuori dall’applicazione, con la conseguenza che i model risultano privi di una reale logica applicativa. Poi all’esame che raccontate?

19. Può accadere che il prof. De Angelis richieda di visualizzare lo **Use Case Diagram** e domandi di illustrare gli **internal steps** di uno specifico caso d’uso.

Una dinamica tipica può essere la seguente:

**De Angelis:** «Mostrami il tuo Use Case Diagram.»

**Candidato:** «Certamente, eccolo.»

**De Angelis:** «Quale caso d’uso mi vuoi mostrare?»

**Candidato:** «Questo.»

**De Angelis:** «Bene: mostrami gli internal steps di questo caso d’uso.»

**Candidato:** «Certamente, eccoli.»

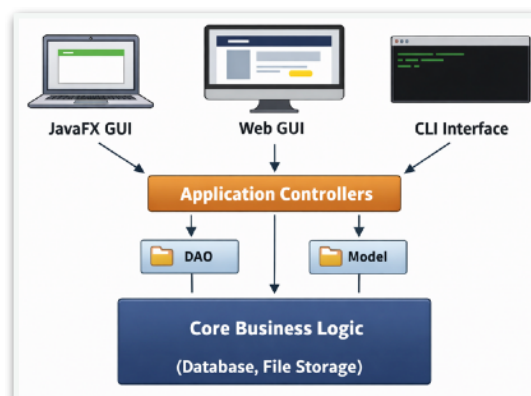
**De Angelis:** «Mi aspetto che questi internal steps siano implementati nel controller applicativo, poiché è il controller applicativo a realizzare gli internal steps di un caso d’uso.»

20. Non devono essere eseguiti controlli sintattici nello strato di persistenza. Tale responsabilità non appartiene alle componenti di persistenza e, qualora venga rilevata, può comportare una penalizzazione estremamente severa (**fino alla bocciatura immediata**).

21. È consigliabile implementare un sistema di caching in RAM: oltre a essere una buona pratica progettuale, è un aspetto spesso apprezzato in sede di valutazione. Qualora lo si realizzi e lo si sappia presentare correttamente, può costituire un elemento fortemente valorizzante. Ad esempio, è possibile integrare il caching con il pattern Observer e un Singleton implementato tramite inner static class, vedete il mio codice.

22. Nella realizzazione della CLI è necessario riscrivere esclusivamente i controller grafici, non i controller applicativi. Se emerge la necessità di riscrivere anche i controller applicativi, ciò indica un errore progettuale (**tipicamente una separazione non corretta delle responsabilità**) e richiede una revisione dell’architettura e del codice.

23. La logica del cambio boundary (quindi delle GUI) deve lasciare invariati i controller applicativi:



24. Personalmente, non ho fatto uso del testo consigliato dai docenti per la preparazione dell'esame, superato al primo tentativo; si tratta tuttavia di una valutazione soggettiva, non necessariamente generalizzabile.

25. Ci saranno giornate dedicate alla risoluzione di problemi o allo sviluppo di soluzioni che, successivamente, magari manco presenterete in sede d'esame a causa di un cambiamento delle scelte progettuali, e quindi perderete tempo inutilmente.

26. Per quanto riguarda il metodo di studio, non esiste una risposta univoca: esso dipende dalla tipologia di contenuti da affrontare.

I design pattern **non devono essere memorizzati meccanicamente**, bensì compresi nel loro funzionamento. È utile semplificare il codice presentato a lezione e realizzare implementazioni ridotte o stub (vedi mio materiale in questo drive sui DP), al fine di chiarirne il comportamento. È fondamentale saper implementare autonomamente un design pattern partendo da zero, almeno una volta.

La parte del corso relativa alla progettazione, trattata dal prof. Falessi, è accessibile ma non deve essere sottovalutata: le lezioni sono sufficienti, ma richiedono comunque un lavoro di riflessione e approfondimento personale.

27. Negli internal steps dei casi d'uso è necessario rappresentare un'interazione bidirezionale tra attore e sistema: non devono configurarsi come un processo unidirezionale.

28. Qualora, dopo un periodo di studio significativo, un design pattern (ad esempio l'Abstract Factory) risulti ancora poco chiaro, ciò indica la necessità di **intensificare in modo sostanziale la preparazione**; tali pattern possono essere utilizzati come parametro di autovalutazione.

29. La prova scritta prevede quattro quesiti: due di progettazione e due di logica, tipicamente relativi a bug regressivi e coverage. La domanda sulla coverage non è sempre presente.

30. Non è richiesto il rispetto del vincolo dell'80% di duplicazione imposto da SonarCloud.

31. SonarCloud deve essere portato a zero; è fortemente consigliato intervenire progressivamente durante lo sviluppo del progetto, poiché un intervento concentrato nella **fase finale** può richiedere tempi **significativamente maggiori**.

32. È normale rimanere incerti sulla qualità della progettazione fino al giorno dell'esame.

33. La correzione della prova scritta avviene immediatamente al termine della prova. Contestualmente iniziano i colloqui orali con il prof. De Angelis, mentre il prof. Falessi procede alla correzione degli elaborati scritti chiamando i candidati progressivamente.

34. Qualora la valutazione della relazione sia inferiore a 10, **non è consentito sostenere l'appello.**

35. In caso di valutazione pari a 0 nella relazione, non è possibile sostenere l'esame nella sessione corrente; l'accesso viene rimandato alla sessione successiva.

36. Ottenere il punteggio massimo con il prof. Falessi, sia nella valutazione della relazione sia nella prova scritta, è facile; tuttavia, la probabilità di conseguire un esito positivo è comparabile a quella di ottenere un esito negativo.

Andare bene è facilissimo.

Andare male è facilissimo.

37. È opportuno evitare di rivolgere ai docenti domande che sottintendano una richiesta di anticipazione o di correzione dell'elaborato. Tali chiarimenti non vengono forniti: le eventuali criticità emergeranno esclusivamente in sede d'esame.

38. È consigliabile valutare con attenzione quali lezioni seguire, poiché alcune risultano in larga parte ridondanti rispetto ad altre e possono essere studiate in modo autonomo senza una significativa perdita di contenuto.

39. Nell'analisi effettuata tramite Sonar è necessario escludere le componenti grafiche. L'aspetto visivo dell'applicazione non è oggetto di valutazione: è possibile presentare un'interfaccia estremamente minimale, anche priva di CSS o framework come Bootstrap. L'attenzione del prof. De Angelis è rivolta esclusivamente al codice applicativo.

40. L'esame orale con il prof. De Angelis può spaziare su un insieme molto ampio di argomenti. Oltre alla revisione del codice, può essere richiesto di correggere o migliorare il VOPC qualora presenti errori, di fornire esempi di modellazione, di scrivere un design pattern da zero o di illustrarne le proprietà. Possono essere richiesti i principi GRASP, i concetti di polimorfismo e incapsulamento, il principio di sostituzione di Liskov (LSP) nell'ambito dell'ereditarietà, nonché la motivazione delle scelte progettuali adottate.

Può inoltre essere richiesto di scrivere in tempo reale esempi di codice (ad esempio per il polimorfismo), di spiegare le differenze tra classe astratta e interfaccia, tra operazione, metodo e messaggio, tra override e overloading, oppure di chiarire il motivo dell'uso di un determinato model in un punto specifico del codice. Possono infine essere chieste le differenze concettuali tra model e bean.

L'esame orale deve pertanto essere affrontato con una preparazione completa e trasversale: è richiesta una conoscenza solida e approfondita di tutti gli argomenti trattati nel corso.

